

RecipeFinder Project Spec

General Description of the Project

RecipeFinder is a Python application that helps users decide what to cook based on ingredients they already have. Sometimes people have food at home but do not know what meals they can make with it. Instead of searching for recipes one by one, the user can enter the ingredients they currently have, and the program will suggest recipes that match those ingredients.

The main goal of the project is to make recipe searching simple and useful. The user will enter several ingredients, such as chicken, tomato, eggs, or rice. The program will compare the user's ingredients with the ingredients needed for different recipes. It will then show recipes that the user can make or recipes that are close matches.

The first version will use a small collection of recipes stored in the program or in a local data file. Each recipe will include a recipe name, a list of ingredients, and simple cooking instructions. The program will focus on ingredient matching rather than using a large online recipe database.

For the first version, the program can run through a command line interface (CLI). The user can type ingredients and view recipe recommendations as text. If time allows, a simple graphical interface could be added later to make the application easier to use.

Task Vignettes (User Activity “Flow”)

1. Enter available ingredients

A user wants to make dinner but does not want to buy more groceries. She checks her kitchen and finds chicken, tomatoes, eggs, and rice. She opens RecipeFinder and enters these ingredients into the program.

The program receives the user's input and separates it into individual ingredients. The ingredients are stored so that they can be compared with the recipes available in the program.

Technical details:

- Receive ingredient names from user input.
- Separate multiple ingredients into a list.
- Clean the input by removing unnecessary spaces and making capitalization consistent.
- Store the ingredients for recipe matching.

2. Find matching recipes

After the user enters her ingredients, RecipeFinder compares them with the ingredient lists of the available recipes. For each recipe, the program checks how many required ingredients the user already has.

The program displays recipes that match the user's available ingredients. A recipe that uses more of the ingredients the user already has should be shown as a better match. If the user is missing some ingredients, the program can also tell her what is missing.

Technical details:

- Store recipes and their required ingredients.
- Compare the user's ingredient list with each recipe.
- Calculate how many ingredients match.
- Identify missing ingredients.
- Display suitable recipes based on the matches.

3. View a recipe

The user chooses one of the suggested recipes. RecipeFinder displays the recipe name, the ingredients required, and simple cooking instructions. The user can then decide whether she wants to make the recipe.

If no suitable recipe is found, the program tells the user that there are no good matches and allows her to try again with different ingredients.

Technical details:

- Allow the user to select a recipe from the results.
- Retrieve the selected recipe information.
- Display the recipe name, ingredients, and cooking instructions.
- Allow the user to perform another search.

Technical “Flow”

The main data in RecipeFinder will be the user's ingredient list and a collection of recipes. Each recipe will contain a name, required ingredients, and cooking instructions.

The basic data flow will be:

User input → Process ingredient list → Compare with recipe data → Calculate matches → Display recommendations → User selects recipe → Display recipe details

The program will need several main blocks:

- **User Input:** receives the ingredients entered by the user.
- **Input Processing:** separates and cleans the ingredient names.
- **Recipe Data:** stores recipe names, ingredients, and instructions.
- **Recipe Matching:** compares the user's ingredients with each recipe and determines which recipes are suitable.
- **Recommendation Output:** displays matching recipes and missing ingredients.
- **Recipe Details:** displays the information for the recipe selected by the user.

For Version 1, recipe information will be stored locally rather than requested from an external API. The application will use basic Python data structures such as lists and dictionaries to organize the information. User interaction can be handled through the command line. A later version could use a larger recipe dataset, an external recipe API, or a graphical interface.

Final Self-Assessment

The biggest change from my initial sketch to this project spec is that the idea has become a more concrete implementation plan. I now have a clearer understanding of the project logic, the steps the user will follow, and how the program will process the information.

I feel confident about implementing this project because I plan to use Python concepts that I have already learned in class. However, my background is mainly in design, and this will be my first time independently developing a complete programming project. I think the main potential challenge may be designing and implementing the recipe matching logic, including how to organize the recipe data and compare it with the ingredients entered by the user.

I may need help from the instructor to check whether my implementation plan is reasonable, especially the recipe matching process. After completing Version 1, I may also need some guidance on how to improve the project and plan the next version.